

四川大學

《软件工程社会职业问题》



期末作业

专 业 软件工程

姓 名 Gother

学 号

指导老师 刘 宇

成绩分数

二零二五年六月一日

题干描述：

低空飞行安全是低空运行的前提条件。项目团队要开发一个低空飞行器冲突检测与解脱系统。主要功能是检测飞行冲突、生成解脱方案并发送信息，一种方式是发送给飞行器，由机载系统自动执行解脱；一种方式是发送给飞行器控制人员，由他们发出控制指令执行解脱。

根据材料完成以下问题。

- (1) 请识别出系统的关键性能参数；
- (2) 你会选择哪一种方式发送信息？请结合软件工程相关社会和职业问题给出理由。
- (3) 如果飞行器有机载避障功能，系统生成的解脱方案与飞行器机载避障功能生成的解脱方案存在冲突，你如何处置？请给出理由。
- (4) 结合本项目软件工程过程，分析每个阶段面临的社会和职业问题，并给出解决方法。（不少于 3000 字）

(1) 请识别出系统的关键性能参数

系统设计的总体方向是追求“快、准、稳、广、通、明、全”，这将为低空智能空域筑牢基础。

[1] 实时性 T (Timeliness)

低空飞行器飞行时速度较快，反应时间短，在城市这种空域环境复杂且人员密集的区域，一旦系统响应出现延迟，就可能引发严重甚至无法挽回的碰撞事故，这类系统对时间敏感性要求极高。

[2] 准确性 (Accuracy)

系统要在多目标、动态环境里保持高识别精度并保证解脱有效，虚警率需控制在 $\leq 0.1\%$ ，漏检率要 $\leq 0.01\%$ ，系统生成的避障或脱离路径成功规避冲突的概率应 $\geq 99.9\%$ ，系统要平衡复杂性与实用性，识别要准确，不能频繁出现误报、错报情况。

[3] 可靠性 (Reliability)

系统运行稳定性是飞行安全的保障，需有较强抗故障能力以及通信链路的持续可靠。飞控软件，要达到航空级认证标准，有高度稳定运行能力和容错特性。

[4] 鲁棒性 (Robustness)

系统设计要考虑非理想工况下的稳定性，比如在网络不畅、数据不全、传感器功能异常、信息精度下降等状况下，系统仍要维持核心功能正常运转，在复杂或不可预知环境中，也应有一定容错能力，防止因局部故障影响整体

运行安全。

[5] 覆盖范围与容量（Scalability & Coverage）

随着城市低空空域逐步开放，未来的运行环境将趋于高密度化。因此，系统应具备处理大规模飞行器同时运行的能力，并支持数量级在数千以上的并发飞行器管理。

[6] 解脱方案合理性与安全性（Resolution Validity & Safety）

生成的避障、规避方案要兼顾技术执行性与现实合规性。不仅要避开禁飞区、航线冲突区，避免高代价机动，还要方案需根据飞行器性能参数动态生成。不能仅凭数学最短路径，需结合飞行器“能飞”的实际能力。

[7] 互操作性（Interoperability）

未来空域不是单一厂商封闭系统，而是开放互联的复杂体系。低空飞行器系统高度异构，互操作性决定了解脱方案能否准确、及时执行。系统应可与不同厂商、不同类型的飞行控制系统、地面控制站、UTM 平台无缝对接，支持与多种飞行器型号和控制系统的对接，接口协议统一。

（2）你会选择哪一种方式发送信息？请结合软件工程相关社会 and 职业问题给出理由。

我更倾向于首先将冲突解除指令直接传送到飞行器本身，让其机载系统自动执行，同时把人工介入设为备用方案，原因如下：

[1] 社会责任与公众安全导向

在低空城市环境中，飞行器飞行高度低，周边空间拥挤，预留的应对时间短，这种环境下，依赖人工操作有天然劣势，自动系统能缩短反应时间，实现更高效的避障行为。采用自动解脱机制可减少因疲劳、经验不足或认知负荷高导致的操作偏差，提升系统整体安全系数。

[2] 软件工程伦理与设计原则考量

这种方式更符合可预测性与形式化验证的工程要求，自动化控制流程可凭借仿真测试、形式化建模提前验证，保证每种情形下的响应路径有依据，依赖人为判断的流程往往不可控，无法充分测试和预判，违背工程可靠性要求。自动执行路径更具可控性与可问责性，体现工程师对“可交付、安全可持续”的专业承诺。

[3] 系统普适性与未来扩展的趋势适配

随着飞行器技术进步，越来越多机型有自主飞行能力，不再依赖地面操作员持续遥控，在这种趋势下，自动执行机制更易与飞行器的飞控系统深度整合，实现闭环控制与自主决策。

采用自动执行方案还可系统在整体架构上的扩展性与模块化集成。它更容易地对接上层空域调度平台、边缘计算节点等基础设施，实现空域资源的动态管理、航线优化以及集群式协同避障等复杂任务。

[4] 推荐机制：保留人工介入作为冗余备份

尽管自动解脱是首选，出于风险控制与训练需要，系统仍应有一定“人工确认或介入”能力。



图 1 人工介入系统示意图

在非紧急场景下，系统可先下发解脱方案至控制人员界面，由其确认后执行，比如训练、试验飞行。在方案不确定性较高或伦理敏感场景，如载人飞行器，可设置“人工干预优先”模式；

建议构建一个“威胁等级与响应权限”相对应的机制框架，在高危状态下，如需毫秒级响应，由系统直接自动执行，中等风险情境下，可预设应对方案并保留人工干预的接口，而在风险较低的情形中，则主要依赖人工决策，系统仅作为辅助信息提供者。

(3) 如果飞行器有机载避障功能，系统生成的解脱方案与飞行器机载避障功能生成的解脱方案存在冲突，你如何处置？请给出理由。

我的观点为融合优先、动态仲裁、主从协同。

当系统端给出的解脱路径和飞行器机载避障机制出现冲突时，不能只采用某一方的判断，而是要引入融合分析与冲突裁决机制，一同完成合理且安全的最终决策，具体建议及理由如下：

[1] 融合判断优先——利用全局+局部信息联合决策

一般情况下，地面平台提供的解脱路径是依据宏观空域信息综合计算而来，飞行器机载系统主要依靠本地传感器感知周围的即时变化，更关注当下实际风险因素。建议系统设置双向通讯机制，让飞行器能把自身生成的避障策略反馈给地面。

[2] 建立冲突仲裁机制——设计自动判优逻辑或人工介入通道

为协调两套可能存在差异的方案，可设计基于风险成本评估的仲裁机制，

对每个方案从安全性、代价、执行可行性等维度打分，最后选出最优解。还可配置优先级规则，比如在紧急状况下，优先执行飞行器自身避障方案，若冲突处于远距预测阶段，则采用系统计算的方案，若风险等级不明或判断结果模糊，可将决策权交给人工介入。

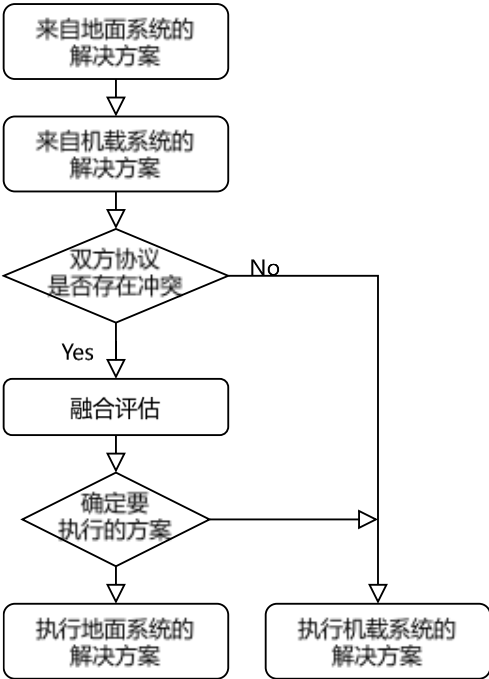


图 2 融合判断+仲裁机制流程图

[3] 构建主从协同机制——明确系统与飞行器的控制主权边界

可将系统与机载控制关系设计成“主控-备控”结构，明确各自的触发边界与退让机制。

[4] 保证可解释性与责任可追溯性

系统要对所有解脱路径的生成过程、冲突识别以及最终选择的依据，建立完整的结构化记录，并添加明确的责任标识，建议构建一种由人类操作、飞行器自主判断与地面系统调度三方共同参与的协同管理体系，根据不同风险场景和时限要求，灵活分配主导权，打造“自动效率、人工智慧与系统整合”三者互补的控制结构，实现动态响应和稳健运行。

（4）结合本项目软件工程过程，分析每个阶段面临的社会和职业问题，并给出解决方法。（不少于 3000 字）

嵌入智能空域的良知

——低空飞行器冲突检测系统的软件工程伦理实践

一、引言：当智能进入空域，良知应嵌入系统

当代空域正经历着一场意义深远的变革，随着低空管理政策持续推进，围绕“空中控制权”的重塑已在全球范围内悄然拉开帷幕，这一技术变迁为我们勾勒出一幅极具未来感的城市画面：快递可借助无人机迅速投送至居民楼顶，灭火无人机可径直飞向火情现场，整个城市空域摇身一变成为有序运转的三维交通网络。

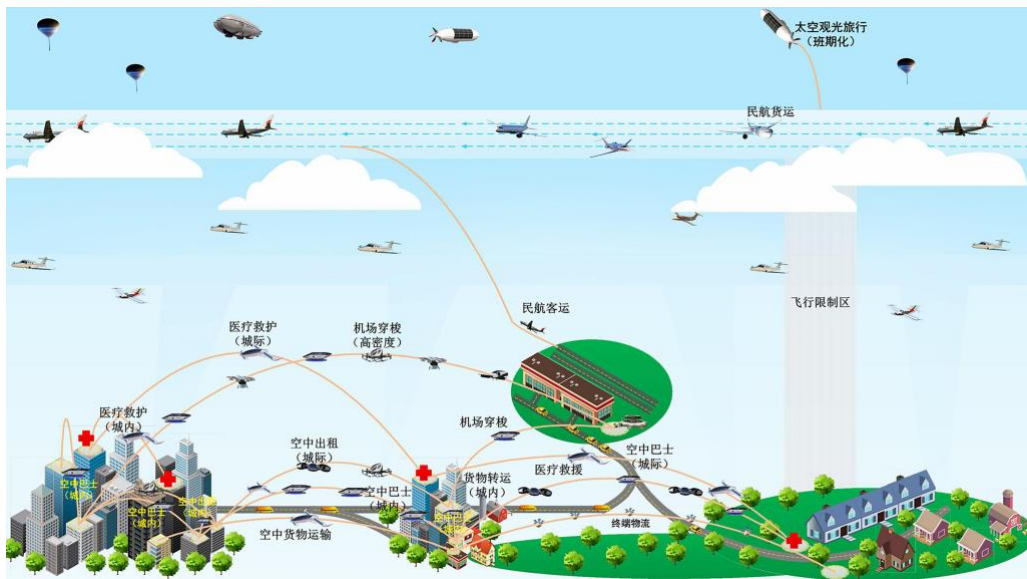


图 3 空中交通智能化运行概念场景

然而，通往这一美好愿景的道路并非一帆风顺，城市低空从本质上来说是一个受限、动态且复杂的多主体空间，在这样的环境中，一架飞行器的飞行路径有可能与多架其他飞行器存在交汇点，稍有偏差便说不定引发严重后果，如定位偏移、传感器故障以及算法错误等，有强大冲突预警与化解能力的系统成为保障空域安全的关键所在。

但我们要清醒地意识到，这类系统的建设并非只是技术指标的简单堆砌，也不只是功能的完善，它实际上是一次关于社会结构与责任分配的深度对话，涉及技术层面，更关乎信任、规则以及伦理底线。

本文并非仅仅聚焦于冲突检测系统的效率与性能，而是尝试从软件工程全流程的视角出发，审视一套智能系统如何在算法、架构与更新过程中融入人类社会的价值秩序，如何在每一次路径计算中体现对生命、责任与公平的尊重。

我们坚信，真正值得信赖的天空系统，要有智能，更要有良知，而良知，应当被融入每一个函数、每一个协议以及每一个自动决策之中，成为城市空域里必

不可少的“道德算法”。

二、从“架构”到“良知”：智能系统背后的五重伦理挑战

构建一套用于城市低空管理的智能系统，并非仅仅是解决一系列技术及算法方面的难题，实际上它更像是一个持续推进的责任故事，从最初的需求分析、系统架构设计，一直到最终的测试以及部署，每一步的背后都隐含着价值观的抉择，许多看似纯粹的“技术问题”，实际上都承载着对社会影响的预先设定，其结果可能会直接对使用者的信任、安全以及公平产生影响。

在需求阶段，开发团队大多时候需要在多方利益之间进行权衡，运营商更加看重调度的灵活性以及成本的控制，监管机构则侧重于规则的执行以及空域的安全，而普通市民则更为关注隐私权的保护以及噪声控制等生活质量方面的问题，要是一个系统的设计过程仅仅听取技术或商业主导方的意见，却忽略普通公众的需求，那么其最终产出的算法结果很可能会呈现出结构性的不公。

在设计阶段，系统初步获得了对行为的“建议权”，它不再只是被动地响应命令，而是开始对飞行路径给出判断，当路径规划依赖于不透明的黑箱模型生成时，我们是否还可对其结果进行追问呢？倘若一些老旧型号的飞行器因为无法适应复杂策略而被自动排除在主流程之外，这是否构成了对技术弱势方的无声放弃呢？

而在编码阶段，责任的边界变得模糊。每一行代码当中，可能隐藏着一处补丁逃避、一处异常静默，更为严重的是，一旦关键模块被外包或者黑箱封装，系统的道德锚点将会不复存在。

测试阶段的重点不应该仅仅停留在验证系统在正常条件下能否运行顺畅，而更应该关注其在极端情境中的稳定性，一个没有经过高压测试或者边界条件验证的系统，即便在理想天气下表现出色，也有可能在信号不稳定或者突发气象情况下出现故障。真正值得信赖的系统，要可在最糟糕的时刻依然维持基本的运作，

而当系统投入运行时，真正的考验才刚刚开始：自动决策出现了事故，由谁来负责呢？飞行器偷偷被升级，用户是否知晓呢？一旦信任被消耗，哪怕是最强的算法，也难以赢得社会的共识。

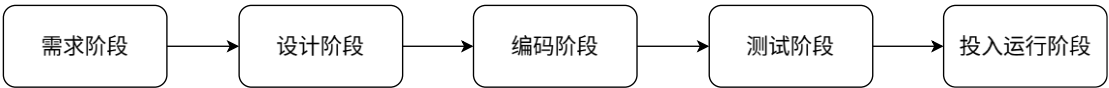


图 4 软件生命周期的五个基本流程

以下五章将按照软件工程生命周期，逐一揭示智能冲突解脱系统中被忽视、被默认、被接受的伦理难题，并尝试提出一套可兼容效率与公平、性能与解释、自动化与人本之间张力的设计回应。我们期望，在理性架构的每一层之下，都可

看到某种形式的“道德预设”在隐秘地支撑系统的可信。

三、需求阶段：在多元诉求之间识别公共价值

传统的软件开发过程中，需求分析通常聚焦于功能规范和系统边界。但一旦系统深入参与城市低空运行，承担起对具备自主控制能力的飞行器进行调度的任务，需求的含义便不再只是“功能性清单”。它实际上成为一项围绕公平、权利分配和价值取向的公共讨论，涉及的不仅是技术人员，更包括社会各界的广泛参与。

3.1 多方利益冲突：技术规范背后的博弈剧场

低空冲突检测系统涉及对象众多，其服务对象包括飞行运营方、监管部门以及公众等，在制定需求时，需要在多种立场间找寻平衡，该系统设计之初处于多利益方交汇点，这使得需求阶段有复杂性与争议性。

冲突检测与解脱系统面对的是多维用户群体，其需求天然存在矛盾：

- 企业用户追求效率与灵活性，期望系统拥有路径重构等高效策略，
- 管理机构注重秩序与可控性，强调行为可追溯、规则可验证，
- 普通公众虽非系统直接操作者或使用者，但其日常生活可能直接受系统运行影响或保护。

这三类诉求存在内在矛盾：比如，提高检测精度可能增加计算负担与成本，若为降低误报率，系统可能过度采取冗余规避措施，影响运行效率，对海量飞行器的调度可能牺牲个体隐私，若需求制定时只关注技术可实现性或经济收益最大化，系统易成为强势利益方服务工具，失去公共性。

3.2 伦理风险与盲区：那些未被建模的人与场景

软件开发中有个危险假设叫“主流用户建模”，当以平均场景设定系统行为时，会将边缘使用者与边界环境排除在设计视野外，结果是系统运行稳定但不公平，功能运行正确却未照顾到所有需要它的人。

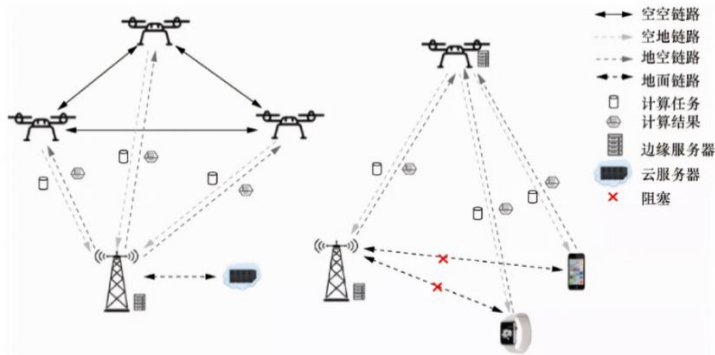


图 5 智能空域的边缘化图示

这样的盲区在飞行系统中并不少见。例如一些老旧飞行器无法运行最新协议或承载复杂算法，一些地区网络状况不佳，一些小型运营商设备预算有限……若

默许这些设备无法接入系统，是否意味着低空空域只属于大厂、强者与都市？这是数字时代新的“结构性边缘化”。

这种“看不见”的排除发生在人群维度，也出现在环境层面。若设计只考虑“阳光飞行”，即晴空、稳定、可控场景，那么一旦遇到暴雨、干扰、故障，系统便进入“未定义状态”，而真正考验系统责任感的，并非其在理想状态下的表现，而是在最坏情境中是否仍尝试回应与保护。

四、设计阶段：让算法对人类世界保持敬意

在整个开发流程里面，设计阶段属于系统结构以及行为逻辑的关键环节，它从数据路径、模块分工，一直到协议规范、算法配置等诸多方面，为系统的运作奠定了基础，虽说在这一阶段系统还没有上线，不过设计过程中做出的各种决策，已然暗含了对于用户权益、风险分配以及社会影响的基本立场。在面向城市空域的智能系统当中，这些架构决策所决定的不只是“运行机制”，是对于“人”的态度。

4.1 黑箱决策的风险：当“选择”不再可解释

当下，许多冲突检测系统已经开始集成像深度学习与强化学习这类先进算法，依靠这种方式来提升其在复杂环境下的规划灵活性以及适应能力，但是这类算法模型大多在封闭结构之内运行，也就是所谓的“黑箱系统”，使用者很难清楚地知道系统是怎样从输入数据推导出输出结果的。

举个例子，当系统建议飞行器向右转弯而不是左转，或者选择下降而不是保持悬停的时候，其背后的判断依据可能来源于训练样本、参数权重，甚至是模型学习过程中的微小偏差，对于使用者而言，这些决策往往难以预测，又缺乏解释，一旦这种决策引发飞行风险，比如误入限制区域或者避障失败，责任归属就会陷入“系统自动决定”的灰色地带。

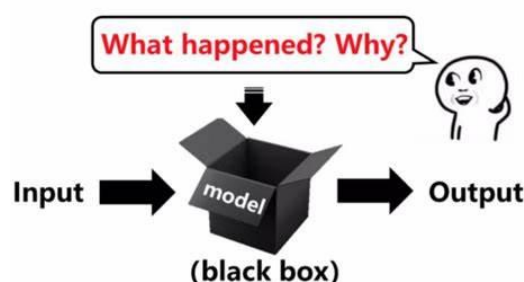


图6 黑箱决策的通俗示意图

在系统设计过程中，要重视用户“理解权”的实现，引入有可解释性的人工智能技术，建立清晰的判断链条，提供基于数据和逻辑的决策说明，不只是为了提高系统的可控性，是在程序层面体现对用户的尊重，

4.2 技术适配偏差：默认谁被接入，实际就是默认谁被排除

冲突检测系统大多时候基于对理想设备的假设来进行设计：高精度定位、强算力平台、实时数据链路。然而在现实中，许多飞行器算力有限、接口老旧、通信延迟不可控，这就致使系统在默认逻辑上“天然排斥”这些设备，

从表面上看，这种基于技术能力的判断好像是客观中立的，但实际上，它构建了一种结构化的不平等体系：高端设备因为其强适配性，享有更多路径选择以及更低的受控概率，而配置较低的设备被限制使用范围，甚至在资源紧张或者应急调度的时候，有可能被系统默认“优先让出”或者“回避处理”，承受更大的运行风险。

4.3 安全与效率的冲突：谁来划定最小保护底线？

效率是推动技术系统发展的关键动力之一，然而当系统管理的对象涉及公共空间、人员安全以及财产保障等高敏感性场景时，效率不应成为凌驾于安全之上的首要考量目标，在此种情况下，需要更加谨慎地去权衡系统响应速度与安全防线之间的关系。

在设计阶段，为了提高路径规划速度而简化避障计算，为了提升系统吞吐量而压缩判断时间窗口，为了减少调度延迟而放宽最小安全距离，然而在这些所谓的“优化”背后，大多时候是以“理论可接受”作为掩护，实际上是将风险隐性地外包给了终端用户。

4.4 工程伦理设计机制：用架构写下系统的责任边界

面对上述风险，提出了三项设计阶段的伦理嵌入机制。

其一构建可解释性接口标准，将每次算法决策结果与对应的解释字段一同并存输出，这样的接口供飞手理解使用，也可供日志审计以及后期责任划分使用。

其二系统应该制定一套“最低运行能力标准”，以此来保障不同类型飞行器在基本功能上的接入公平，不管设备性能高低，都应该可接收简化的规避指令，传回基本状态信息，并且在突发情况下有最低限度的避障能力。

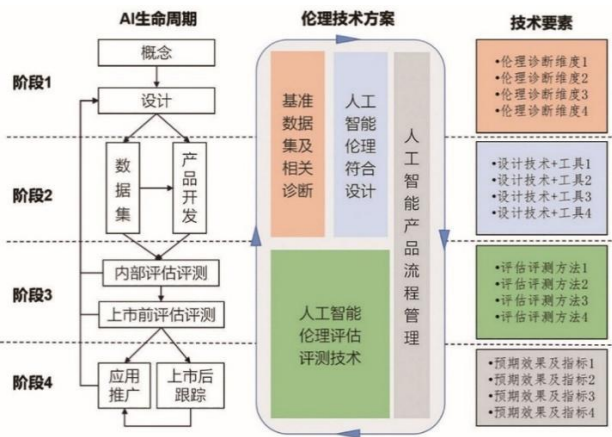


图7 人工智能伦理技术框架

其三内嵌“默认保护优先”结构逻辑，明确系统在冲突处理中优先保护哪些

目标，何时触发人工审批，哪些模块不可自动化覆盖。这些逻辑将以参数化形式写入架构配置文件，成为系统“不可随意更改的伦理常量”。

一个系统是否“智能”固然关键，但是更为关键的是它是否值得信赖，只有当系统在设计之初就明确责任边界，纳入公平规则，才可在未来真正成为承载社会价值的技术载体，而不是仅仅是冷冰冰的自动化工具。

五、编码阶段：在每一行代码中注入责任

代码并非表面呈现的那般中立，它所反映的，除了开发者的技术水准，还包括其对系统后果的判断能力与责任意识，一个看似毫无危害的默认参数，一回因急于赶进度而跳过的错误处理，亦或是一行标注着“待优化”的注释，在未来都有可能成为系统漏洞的起始点，甚至在关键场景中演变成导致事故的诱发因素。

5.1 技术债：从权宜之计到责任缺失

在项目周期紧凑、需求持续变动且测试资源有限的状况下，程序员大多时候会采用“技术债”的方式来应对压力：先编写一版可运行的代码，待有时间时再进行重构，先添加一个补丁作为兜底，等下个版本再做优化，先硬性编码一个变量，等设计确定后再进行抽象。

这些被暂时搁置的代码结构尽管在短期内可项目的推进，然而却大多时候欠缺基本的错误边界检测以及异常容错设计，同时也缺少清晰明了的说明文档与责任归属标识，一旦系统在运行过程中出现异常，很难迅速确定问题的根源所在，排查过程复杂，而且责任认定也会变得极为模糊。

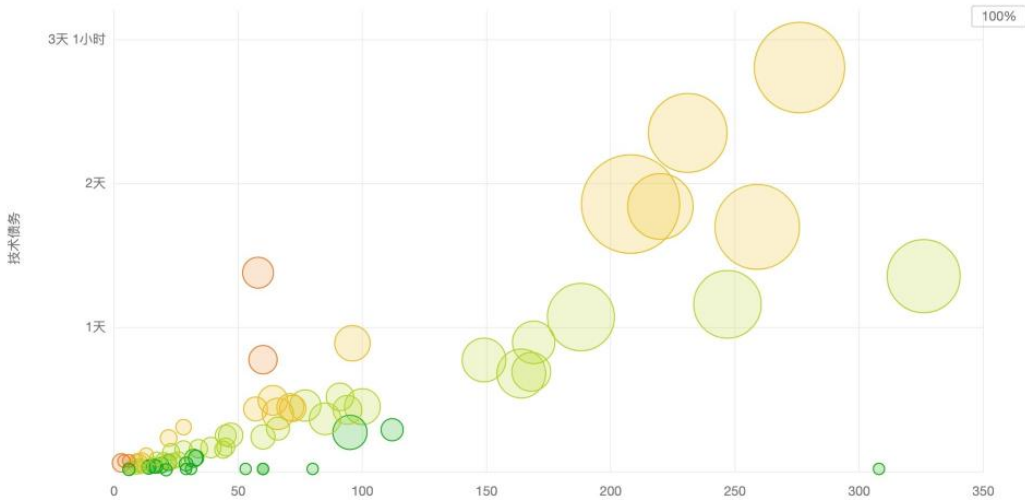


图8 代码“技术债”增长图

真正的职业责任感，并非仅仅是让系统“可运行”，更在于要让它“运行得清晰明了”。技术债从来都不是代码层面的问题，而是伦理层面出现了松动。

5.2 外包模块：系统责任的断裂带

在复杂系统的开发进程中，如路径计算、三维建模、图像识别等高计算密度

的模块，大多时候依赖第三方组件或者开源框架来实现，虽然这种做法在一定程度上可节省开发时间并提升效率，但是也给予了责任链条不清晰的问题，使得系统整体的可控性与稳定性面临挑战。

许多外部模块仅以接口的形式存在，代码不可见、逻辑不可控、数据不可追溯，一旦模块出现异常表现或者与主系统行为发生冲突，原开发者便可能会以“模块责任外包”作为理由来推卸后果，这种结构削弱了系统的整体可解释性，还破坏了软件工程的基本信任架构。

5.3 在代码中写下责任：伦理编程的实践方式

编码阶段的伦理责任，不能仅仅依靠开发者的良心来约束，更需要在工程实践中有具体可行的途径。以下是三项实践机制，用于将伦理原则融入代码结构之中：

其一为结构级的责任注释机制，每个关键的判断逻辑都应当保留理由注释，这是为了让团队成员可理解，也是为了给未来可能出现的事故分析以及责任划分留下清晰的线索。

其二建议在系统调用外部模块的过程里，加入接口层级的行为确认机制。系统应当明确标识每一次调用的责任方、所依据的输入数据以及运行条件，并且对调用结果进行验证或者反馈记录。

其三建议在系统部署之前设置一个专门的伦理审计流程，对核心功能模块进行系统性审核，重点检查是否考虑了边界输入、是否存在日志缺失、是否使用了未经审查的硬编码值等。这一环节是保障代码质量的技术手段，也是防止道德风险的最后一道防线。

六、测试阶段：用边界测试映射道德边界

6.1 从“能运行”到“能负责”：测试的伦理底线

虽说测试阶段一般被当作上线前的技术保障举措，然而要是从社会责任以及伦理角度去看，它事实上是一场针对系统能否获取用户信任的关键演练，即便系统在标准条件下运行顺畅，也不能证明其在极端情形下的可靠性，而实际里真正棘手的风险，往往就发生在那些被忽视、未充分测试的边界情境之中。

6.2 忽视边界场景，就是在回避最真实的失败

真实世界的不确定性，一般在边界场景里最为集中，可是在许多测试流程中，这些场景大多时候被视作“可延后处理”甚至“可忽略”的部分，边界测试之大多时候被压缩、延迟，原因就在于成本高、失败多、回报小，但正是这种逃避行为，致使大量系统表面上凭借了验证，实际上只是避开了真正复杂且脆弱的部分。

一个系统要是无法在最差条件下维持最基本的运行能力，那么即便它在最理想状态下表现得再完美，也只是“假定安全”。

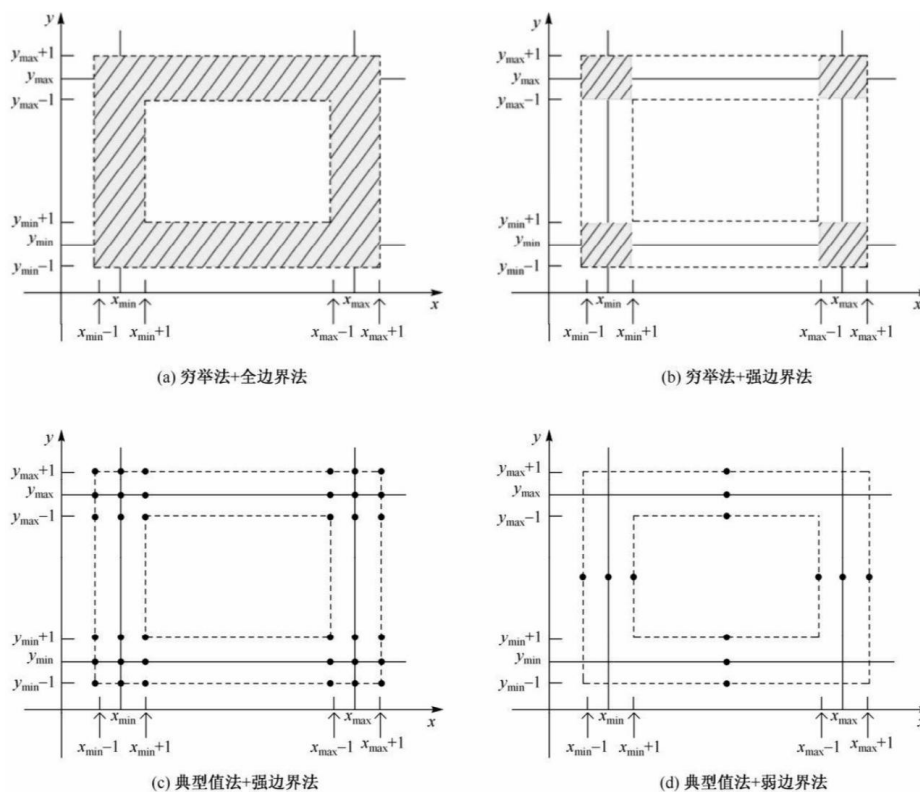


图 9 合理的边界测试方法之一

6.3 为测试嵌入责任逻辑：从仿真到承诺

系统测试环节不只是功能正确性的确认过程，更应当体现出伦理责任的承担，在冲突检测类系统中，这种责任具体可从三个层面来落实：

测试方案中应把极端情境纳入常规流程，而非作为可选附加项。比如在构建测试集时，要考虑突发天气、飞控失联、数据链断开等高不确定性事件，目的并非是为了极限挑战系统，而是要检验其在重压之下的表现，包括是否有收敛失败、降级缓冲，甚至向外部寻求协助的能力。

每一个尚未覆盖的测试场景、每一个未经验证的模块、每一种暂不兼容的设备，都应在用户手册或技术说明中明确注明其风险等级。这可用户做出知情决策，也能防止系统在“默认沉默”中营造不真实的信任氛围。

系统上线前应经历一次“伦理复核”，测试团队需要给自己提一个问题：我们是否已将弱势设备纳入验证？是否模拟过信号不良、感知受限的场景？是否在文档中明确告知用户系统尚未完成的部分？并非每个问题都要被解决，但每一个风险至少都应该被正视与陈述。

七、部署与运行：系统上线之后，伦理责任不应下线

在众多工程项目里，部署一般被视作完成的标志，而运行则被看作是“技术生命周期的末尾阶段”，然而对于面向社会运行的飞行冲突检测与解脱系统来讲，部署并非结束，而是责任开始切实落地的那一刻，从它正式发出指令、对路径产

生影响、触发避障开始，每一个系统行为都会带来实际后果，这些后果涉及飞行安全、公众利益，甚至关乎生命界限。

随着智能飞行技术不断发展，系统的自主决策能力持续提高，它可做出判断，还可以执行行为指令，这也就意味着它的每一个决策都隐含着某种责任承担，一旦发生飞行冲突，我们就不得不去追问：责任归属谁？用户是否清楚系统近期的算法或协议有没有更新？自动操作是否有留下记录？也就是说，系统虽然“上线”了，但它在伦理层面是否也保持着“在线状态”呢？

7.1 自动行为之后，责任不能“默认失联”

一旦系统进入部署运行阶段，其行为会渐渐被当作“常规操作”。冲突判断、路径生成以及避障指令下发，都由系统自主完成，可是一旦自动系统行为引发意外，后续的责任归属大多时候会陷入模糊不清的境地，是操作员使用不当？是硬件出现故障？还是算法判断存在缺陷？在缺乏明确记录与责任机制的状况下，很难准确弄清楚。

要解决责任追溯的难题，关键并非事后争论谁该负责，而是要在系统架构中预先建立清晰的责任链。每一次解脱指令都应附带数字签名和源信息，明确指出该决策的来源数据、所使用的算法版本、输入参数等内容，并且保留执行过程的完整日志记录，借助这样的结构设计，才可实现真正的可审计、可纠错以及可追责。

7.2 系统升级不是技术维护，而是伦理事件

在系统运行过程中，更新与优化是常态，比如通信协议调整、模型参数调试或者路径算法更换等。在一般软件中这属于常规的运维事项，但在自动控制类飞行系统中，每次更新都有可能改变行为逻辑，甚至影响飞行判断标准，这些升级不应仅仅当作技术动作来处理，而应被视为一次次与用户之间“使用协议”的更新，至少要向用户清楚说明改动内容，并预留版本回滚与应急修复的手段。

7.3 不只是稳定运行，而是持续可信

部署阶段的核心，不是系统能否保持功能不崩溃，而是它能否让公众信任不下降，真正的“上线”，不是安装完成或者推送成功，而是系统从一套工具，转变为一个责任承担者的开端。

这类系统一旦上线运行，应持续有四种核心能力：可被质疑、可被理解、可被监管、可被纠正，一旦这些机制在系统维护中逐渐被弱化，虽然系统功能看似仍正常运转，但其伦理责任实际上已经处于。

八、总结：以责任之心构筑未来空域秩序

当下正处于城市低空空域智能化加速推进的关键时期，飞行器所扮演的角色正发生着深刻的改变，它们不再只是单纯用于远距离飞行的特殊工具，而是会逐

步融入日常城市生活之中，承担起如物流配送、城市巡检、紧急救援以及未来载人出行等各种各样的任务，然而要实现这种未来的情景，仅仅依靠性能更为强大的飞行平台以及更加精准的感知设备是远远不够的，迫切需要构建一套稳定、可靠且令人信赖的空中运行秩序体系。

在这样的空中秩序里，飞行冲突检测与规避系统无疑是核心支柱之一，它需要在复杂的信息环境下感知潜在冲突，还要在多变的空域中迅速规划出最优路径，有时甚至需要在紧急状态下替代人为做出决策，倘若这种系统拥有决策权，那么就应当同时承担因其路径选择所带来的相应后果。

回顾软件工程的整个流程，我们可发现，每个阶段都并非仅仅是代码的推进，是良知的嵌入之所，在需求阶段，涉及到谁会被看见，谁会被忽视，设计阶段，思考的是是否尊重人，编码阶段，是在书写责任，测试阶段，要有承认失败的勇气，部署与运行阶段，则要保持对社会的解释力。

当飞行器开始有“选择”的能力时，我们要保证，它在每一次的选择中，都蕴含着人类对生命最基本的敬畏之情。

参 考 文 献

- [1] 朱永文, 陈志杰, 蒲钊, 王琦. 空中交通智能化管理的科学与技术问题研究[J]. 空军工程大学学报, 2021, 22(6): 1 - 10.
- [2] 民航局. 关于印发《低空飞行服务保障体系建设总体方案》的通知[Z]. 民航发〔2018〕100号, 2018-09-28.
- [3] 张风玲, 丁翠玲, 陶艳, 等. 航天型号嵌入式软件边界测试方法研究[J]. 工业控制计算机, 2024, 37(06): 24-26.
- [4] 景文君, 王震, 姜沛妍, 等. 航空软件容器化部署管理技术研究与应用[J]. 航空电子技术, 2024, 55(04): 44-51.
- [5] 刘茂珍, 陈平, 谢梦珊, 等. 开源软件漏洞责任归属问题的剖析与对策[J]. 广东科技, 2024, 33(06): 114-118.
- [6] 郑昭文. 软件外包项目沟通管理评估研究[D]. 东北财经大学, 2023. DOI:10.27006/d.cnki.gdbcu.2023.001454.
- [7] 彭雨松, 王圣, 张磊, 等. 基于生命周期的软件供应链风险分析研究[J]. 网络安全技术与应用, 2025, (05): 59-62.
- [8] 范鑫, 陈贯鑫, 刘宜朋. 探索软件工程伦理在软件工程专业教学中的应用[J]. 黑龙江教育(理论与实践), 2024, (11): 48-51.